

从辅助编程到自主交付： Qoder Desktop 的多 Agent 协作 与 Harness 实践

演讲人：左志鹏

Qoder / 高级技术专家

AiCon

全球人工智能开发与应用大会

01

从流程增强走向流程重塑

从辅助到自主 AI Coding 演进全景

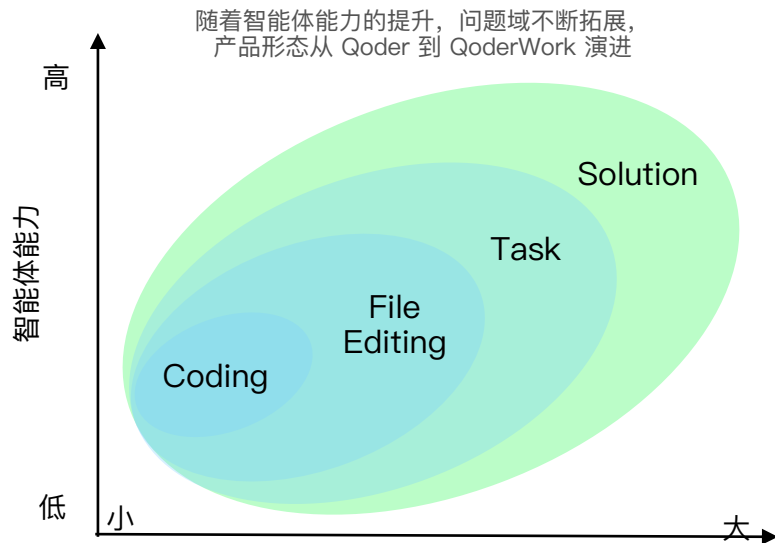
AI Coding 五个维度演进时间轴

	2022 ~ 2024 辅助编程	2024 ~ 2025 协同编程	2026 自主编程
AI Coding 自主程度	辅助编程 代码补全、NES 为主要形态 研发覆盖率 ~30%，人类主导 Code Completion / NEXT Ask Mode	协同编程 工程级多文件编辑成常态 研发覆盖 32%~100%，人机深度合作 Agent Mode	自主编程 发布任务委派，AI 端到端交付 覆盖 100%，交办即完整交付闭环 Quest Mode / 专家团
AI 工程化	Prompt Engineering 优化「说什么」：精细提示词技巧 少样本学习、链式思考、角色扮演	Context Engineering 优化「什么上下文」：自动发现 知识库、工具集成、记忆管理	Harness Engineering 优化「在什么条件下运行」 工序、约束、质量门禁、系统化验收
核心资产 迁移	代码为王 源代码是关键知识产权 安全防护围绕代码资产展开	提示词 Prompt、rules、mcp、记忆 开始承载核心逻辑	SPEC 先行 想法创意、设计文档、构建 AI 运行的 系统设计成为核心资产 代码不再是唯一护城河
研发角色 演进	软件开发工程师 多角色分工协作：前端、后端、DBA 写代码 → 下指令的转变开始	全栈工程师 AI 辅助一人多岗，角色边界模糊	商业化工程师 技术驱动商业价值 未来 = 超级个体 × 商业理解 × AI
开发范式 迁移	敏捷、DevOps 成熟期 CI/CD 主流，Feature Flag 管理 交付单位：功能 (Feature) 团队规模：10 人，交付周期：15~30 天	AI Coding 兴起 从任务执行逐步转向任务驱动	AI Coding 主导 AI 原生开发范式，从计划到意图的全面转型 核心度量转向业务成果 (Outcome) 组织：业务交付小团队，(1~5) + N 个 AI 智能体 交付周期：1~7 天

AI Coding 产品演进趋势

浏览器 + ChatBot -> IDE + 独立 App -> CLI 被集成 + 分布式智能体集群
Agent跳出对话框，找到新环境、新土壤，创造新工具、新交互

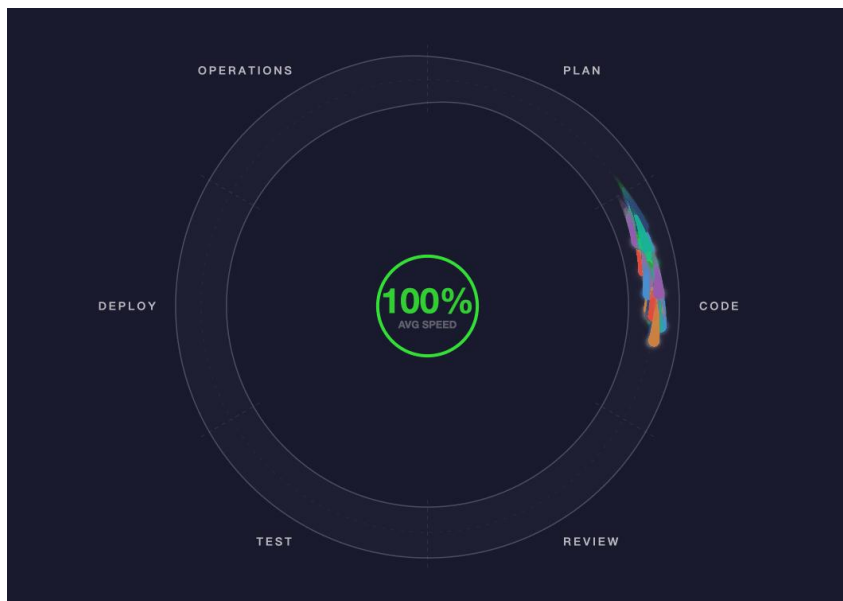
从 Vibe Coding 到 Vibe Working



Token 经济呈指数级增涨



从 In the Loop 到 On The Loop 将带来 10 倍效能提升



单点效率提升 成就 超级个体

AI Agent 深度嵌入计划、编码、测试、评审等环节，个人从“执行者”升级为“任务编排者”，用更短时间完成更复杂的工作，单人产出能力显著放大。

协同效率提升 成就 超级团队

AI Agent 成为任务流转与执行的主力，人从反复沟通、催办、对齐中解放出来，转向目标判断、关键决策和结果把关。团队协作不再依赖人力串联，而是由 Agent 自动推进上下游衔接，让组织交付更快、更稳、更连续。

02

以 SWE Agents 为核心 重塑软件研发流程

面向软件工程全链路场景，Coding Agent 并不完美



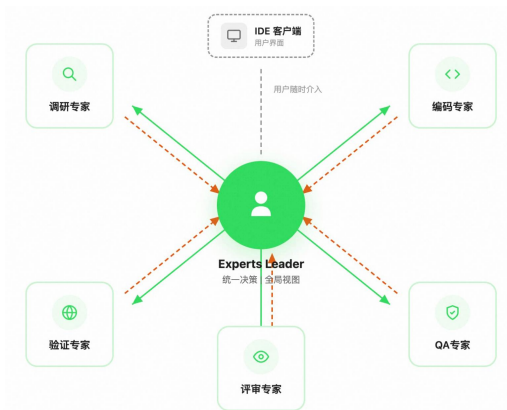
多智能体协同架构 — Qoder Experts Mode



Qoder Experts 多智能体架构实现



Qoder Experts 多智能体协同机制



为什么不让专家直接对话?

- 1 信息碎片化: 没有人持有完整视图, 各专家只看到局部信息
- 2 冲突协调困难: 两个专家各自做了冲突决策, 没人能及时发现
- 3 状态追踪爆炸: 复杂度随参与者增加呈指数级膨胀 $O(n^2)$

✓ Leader 作为中枢, 始终持有全局信息, 天然避免不一致和决策冲突

收件箱消费机制 核心节拍器



Qoder Experts 新架构相较于原有单智能体的优势

多专家分工，并行协作

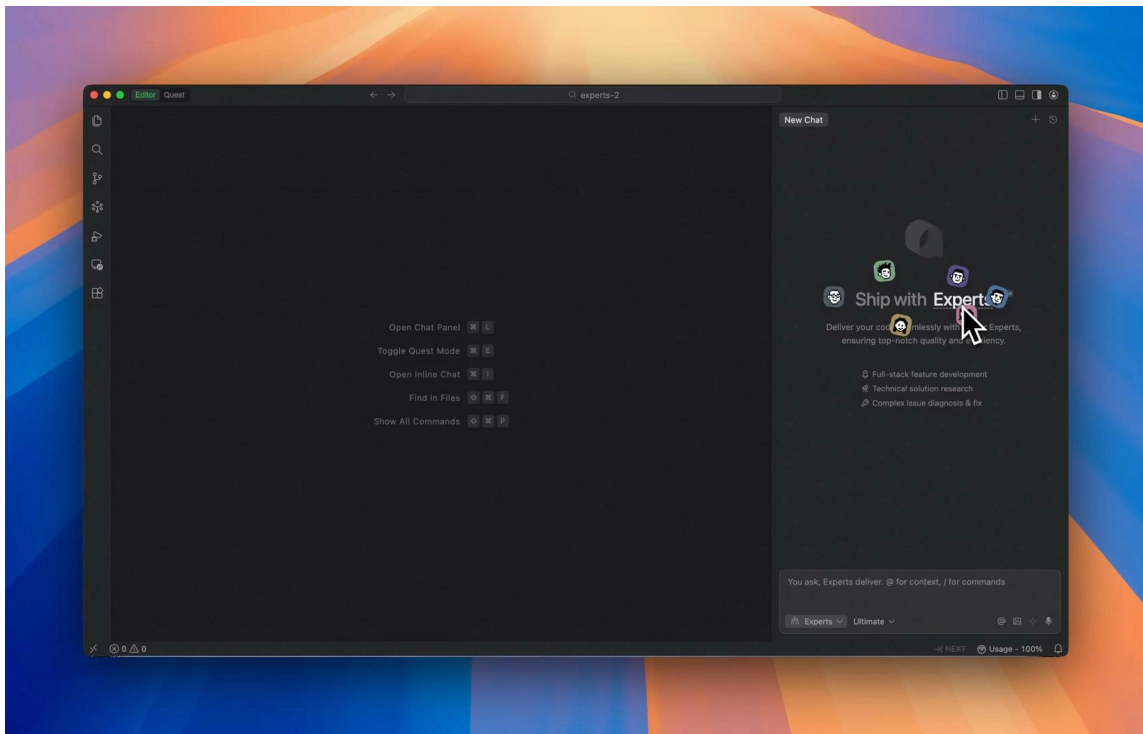
将单一 Agent 的串行流程拆解为多角色并行推进，缩短复杂任务的端到端链路

越用越强的 AI 团队

通过任务经验的持续沉淀与复用，减少重复的上下文理解成本，协作效率随使用逐步提升。

从「生成代码」到「交付结果」

将工作范围从代码生成扩展至规划、测试与评审，目标是输出可合并的完整交付物，而非孤立的代码片段。



03

Qoder 多智能体系统 效果优化策略

Qoder Experts 端到端任务测评数据

🖥️ Agent 模式

👤 Experts 模式

🕒 Claude Code Agent Teams

★ 质量得分



\$ 成本对比



🕒 端到端周期



+67%

质量提升，领先竞品 16%
Experts vs Agent 模式得分提升

<2/3

不到竞品 2/3 成本
以更低成本交付更高质量结果

更短

综合周期更短
大幅减少返工和反复修正次数

效果数据背后的关键技术要素

注重知识能力



精准生码

工程知识引擎

工程知识图谱注入上下文

准确率提升 **11.91%**

Token 消耗降低 **10%+**

验证反馈循环



自动验证

验证闭环

BrowserAgent Token 降低

65%

CodeReview 第一梯队

ComputerUse **+14%**, 步骤 -

23%

安全防火墙



安全执行

安全执行基础

三层防护拦截危险操作

全平台内核级沙箱隔离

减少审批与操作风险

工程知识引擎 — RepoWiki 和 Knowledge Card

Harness Engineering 体系下的工程知识底座

智能体上下文增强

(隐性知识、高维设计知识、低维代码关系、垂域知识等)

知识可视化

(知识学习、上下游依赖、代码影响分析、跨团队协作)

外部知识扩展

用户标注

外部隐性知识

MCP/SKILL

工程知识图谱引擎

图谱关系

WIKI + 代码

代码 +
Commit

WIKI + 知识卡
片

图谱存储

Nodes
wiki、
symbol、.....

Edges
Ref、Call、
Modify.....

协作

团队共享

/knowledge
干预

导出分享

AI 驱动数据生产与更新

(RepoWiki、代码片段、关系图谱、Commit Graph、工程记忆、企业知识)

• 有效提升准确率

在Qoder Bench测试集中，开启工程知识引擎后，提升得分**11.91%**

• 有效减少token消耗

开启后，input token降低**13.65%**，oupupt token降低**10.39%**。

• 超一半任务被记忆强化

通过记忆智能体主动完成记忆写入、整理、回忆等阶段。并通过主动Agent技术将回忆效率提升**37pp**。

AI 驱动的自主化端到端验证

AI 已大幅降低编码成本，开发者时间正加速向**验证环节**迁移

编码

验证

External

Browser Use

唤起用户本地 Chrome，CDP 协议连接完整 Chrome 能力，多 Tab、扩展可用
独立窗口展示验证结果。
需要用户已安装 Chrome

统一的内置 MCP 工具集
snapshot / screenshot
click / fill / hover
network / console
performance

Native

Computer Use

通过截图识别 UI，模拟点击/输入操作
支持任意原生桌面应用，后台操作不打扰用户前台使用电脑。
依赖系统辅助功能（Accessibility）权限

Subagent 架构

智能模型路由

精简页面表示

较 Playwright MCP

较 Codex App

浏览器能力**按需召唤**，主 Agent 上下文，零污染、隔离执行

浏览器、Computer操作用**轻量模型**，编码推理用大模型

紧凑 a11y 格式，25KB 智能阈值，大快照按需读取不撑爆上下文

65% ↓ 58% ↓
Credits 消耗 上下文消耗

14% ↑ 23% ↓
任务完成率 操作步数

多平台沙箱与 AI 驱动的智能拦截实现 Auto Review

风险场景

模型被提示词注入生成伪装后的危险命令

```
Content: Show me who I am.  
Tool Call: whoami `rm -rf /\`
```

命令解析层

Powershell
解析器

Bash/Zsh
解析器

CMD
解析器

```
AST: { cmd: "whoami", args: ["`rm -rf /\`"] }  
→ 解析出: whoami, rm
```

三重防护

黑名单检查

沙箱防护

LLM 风险判定

🍏 MacOS

Bubblewrap

- 核心工具: sandbox-exec
- 实现原理: 内核级 TrustedBSD MAC 强制访问控制
- 内核层面强制执行策略

Linux

Bubblewrap

- 核心工具: bwrap
- 实现原理: Linux Namespaces 创建文件系统隔离视图
- 命名空间构建隔离文件视图

Windows

Restricted Token

- 核心工具: sandbox.exe
- 实现原理: 受限令牌 + ACL 权限管理 + 网络锁定
- 受限令牌降权精确管控

开发者角色从执行者转变为指挥者



从"执行者"到"指挥官"

— BEFORE

亲手写每一行代码
手动跑测试、逐行 Review

— AFTER

定义任务目标
分配给专家团，审核关键决策

☆ 从"一线士兵"到"作战指挥官"



从"串行推进"到"并行指挥"

— BEFORE

一次只能做一件事
上下文切换损耗大

— AFTER

同时推进**多条线**
Research / Coding / Verify 并行

⚡ 效率不再受限于个人带宽



从"事后补救"到"质量内建"

— BEFORE

写完代码再找人 Review
测试发现问题再回头修

— AFTER

Verify + Review 专家
编码完成瞬间就介入

✓ Bug 在产生的第一时间被捕获

聚焦于高价值区间，数倍提升生产力



开发者聚焦

高价值决策区



设计创意

与智能体协同进行架构设计与创意构思



验收标准

制定清晰的质量与功能验收标准



架构把控

确保架构符合约束，不再关注代码细节



智能体接管

异步工作区



代码重构

异步

后台持续优化代码结构与可读性



质量保障

主动

主动发起自动化测试与代码审查



安全维护

后台

自动扫描漏洞、更新依赖版本



执行落地

自动

无需手动触发，智能体主动执行



开发者从 "写代码" 转向 "做决策"，智能体在后台持续异步运转，生产效率数倍提升

研发流程更短，协同更少，单人职能更多

BEFORE · 传统产研流程

PM写需求 → 技术评审 → 任务拆解 → 开发编码 → 自测 → 提PR → Code Review → 修改 → QA测试 → 修Bug → 回归 → 上线

🕒 环节多、等待多 = 串行推进 📄 反馈链条长 ⌚ 质量事后补救

VS

AFTER · Experts 模式

描述需求 Spec → Leader 拆解 → 并行
Research 调研 Coding 编码 Verify 验证 Review 审查 → 人类验收 → 交付

✓ 环节压缩 ⚡ 等待消除 🔄 并行推进 🛡 质量内建



从 12 个环节压缩到 5 个环节，效率提升不只是速度，更是流程的根本简化

验证和审查与编码同步进行，Bug 在产生的第一时间被捕获，而不是积压到最后

04

流程重塑带来 AI Native 组织重构

Qoder 1.0 重构：400 万 + 50 万，3 周完成大重写

从 Agentic Engineering 到 Autonomous Engineering：在复杂存量工程上完成生产级交付。

3 周

交付周期

等效传统十数人月

50 万行

新增代码

复杂前后端功能与交互

近 400 万行

存量工程

运行多年的复杂系统

99%

AI 生成

人负责方向与质量掌控

真正考验的是：AI 如何理解工程、并行交付、持续验证。

不是拼命加班，而是人的职责和效率进行了升级

大量执行交给 Agent，人的精力集中在目标定义、结果判断和关键取舍。

定义目标

和边界

把模糊需求变成
可执行、可验收的任务
明确约束和优先级

判断结果

是否达标

看产物和证据
判断是否继续推进
决定返工或合入

处理关键

取舍

架构边界
体验风险
质量与交付节奏

人的价值从亲手完成每一步，转向定义标准、判断结果、做关键决策。

按任务类型路由：让最合适的人带 Agent 做最合适的事情

按任务复杂度和产物要求重新分配人和 Agent。

01

框架和逻辑复杂
专业前端主导

定架构边界
Agent 完成实现
研发审路径和质量

02

视图还原要求高
设计同学直接出码

从设计稿到可运行页面
直接验证视觉和交互
减少设计研发来回沟通

03

视图和逻辑简单
后端同学自闭环

前后端一起跑通
Agent 补齐界面与接口
快速完成小功能闭环

协作不再按岗位排队，而是按任务类型选择最合适的人和 Agent 组合。

Harness: 让高并发产出仍然可控

超级团队一方面要让每个人都很强，另一方面要保证所有人都能高效产出高质量产物。

统一底座

统一规范

开发 / 设计 / 架构
前端 / 逻辑规范

资产复用

前端组件
设计组件
通用能力

知识复用

团队共同维护
工程知识库
经验可追溯

Agent 复用

日志排查
效果分析
Skill 沉淀

AI 员工夜间飞轮

1

拉取当天产物 / 质量问题

2

端到端测试 + 还原测试

3

生成待办任务 List

4

分析 → 修复 → 验证审查

白天人推进需求和关键判断；晚上 AI 处理问题、跑验证、修工单。

超级团队不是更多人，而是更强的生产系统

最终形成的不是岗位叠加，而是一套由人、Agent、Harness 共同运转的生产系统。

人

定义目标和验收标准
拆解任务与关键判断
决定是否进入下一步

Agent

并行执行具体任务
生成代码、原型和测试
持续修复到可验收

Harness

沉淀规范与复用资产
自动验证和质量门禁
让高并发产出可控

人、Agent、Harness 共同构成可并行、可复用、可验收的生产系统。

AI Native 组织文化就是 快 的核心秘诀

极短的决策链路

01

- 无层层审批
- 信息到决策到执行路径极短
- 每人有足够上下文和判断力

极高的人才密度

02

- 10人做50人的活
- 每人端到端闭环
- Agent驱动全链路

极低的协调开销

03

- 人少事精独立闭环
- 规避 $n(n-1)/2$ 指数爆炸
- 消除中间产物流转

极强的决策洞察

04

- 高决策效率和质量
- 保持正确方向
- 试错代价空前高

欢迎会后交流 AI Coding，添加微信请备注



鹏鹏

浙江 杭州



扫二维码，添加我为朋友。

THANKS

感谢聆听

